Hierarchical Software Architecture

Overview of Hierarchical Architecture

- Software is decomposed into logical modules (sub-systems).
- Modules at different levels interact via method calls or service calls.
- The system is visualized as a hierarchical structure.
- Languages:
  - Procedural (functions & procedures)
  - OOP (methods & services)

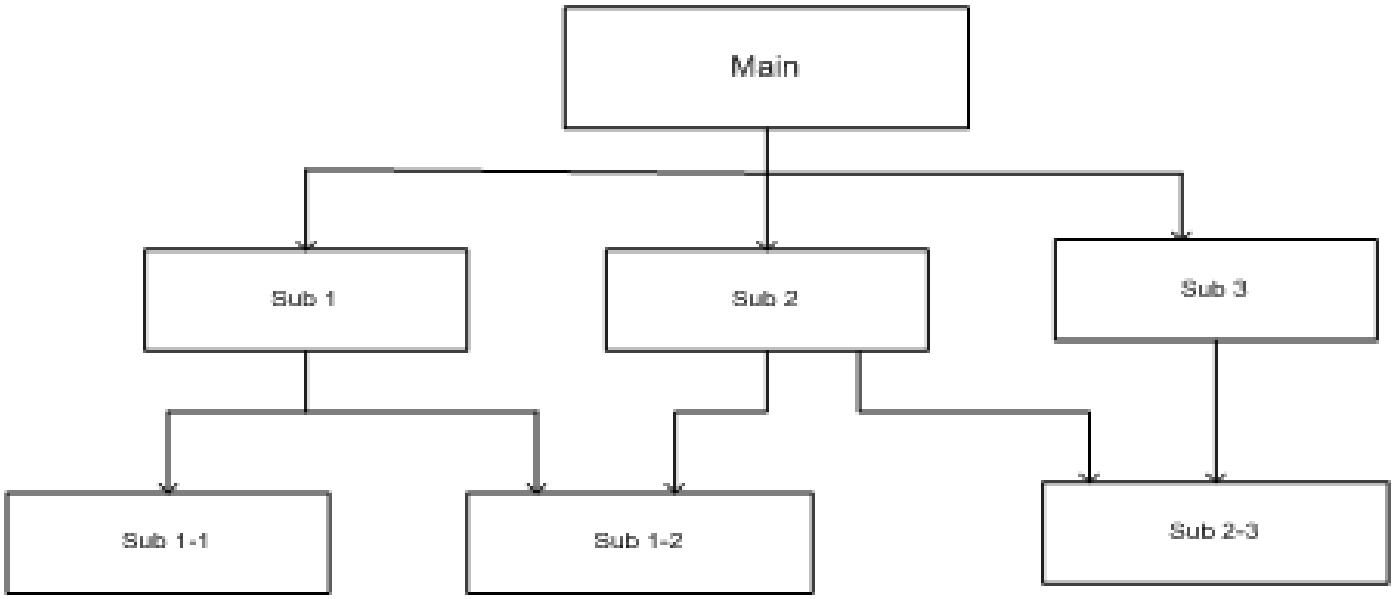
Key Types of Hierarchical Architectures

- Main-Subroutine
- Master-Slave
- Layered
- Virtual Machines

Main-Subroutine Architecture

- Decomposes the system into subroutines, refined hierarchically based on functionality.
- Data Passing
  - Pass by reference
    - subroutines can modify the data.
  - Pass by value
    - subroutines work with copies of the data.

Main-subroutine



Main-Subroutine Example: Payroll System

- Main Process: Payroll Processing
- Subroutines:
  1. Fetch Employee Data
  2. Calculate Deductions
  3. Generate Payslip

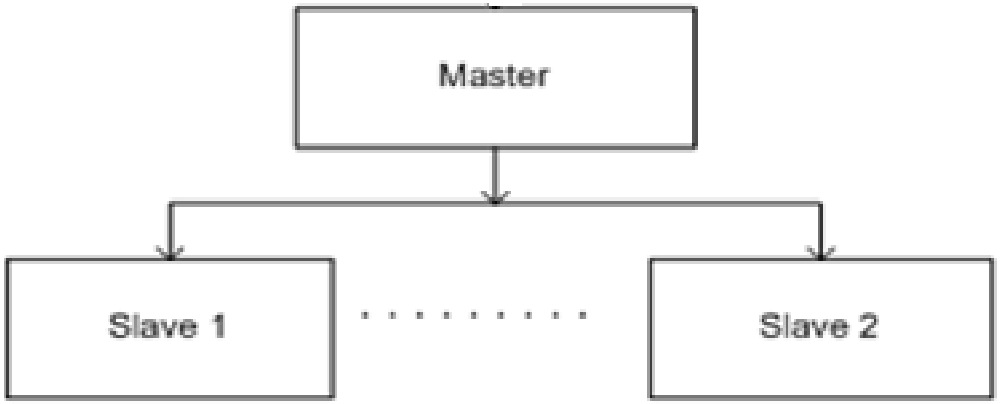
Main-subroutine

- **Benefits**
  - Easy to decompose the system based on the definition of the tasks in a top-down refinement manner

- **Limitations**
  - Globally shared data in classical main-subroutines introduces vulnerabilities.
  - Tight coupling may cause more difficult to changes

Master-Slave Architecture Overview

- Master divides work into subtasks.
- Slaves process tasks and return results.
- Master combines results to generate final output.



Block diagram for master-slave architecture

Master-slave Architecture

- A system must be fault-tolerant(reliable) (i.e., it keeps running even when individual components fail)

  (Example: e-commerce web-site should continue operating even if a database server goes down)

- A system must produce accurate results with high probability, even when individual components fail (e.g., safety-critical systems)

  (Example: Airplane control software must produce accurate results, even if a piece of hardware malfunctions)

Master-slave Architecture

- A system needs to solve *computationally-intensive* problems

  (e.g., problems for which there are no efficient algorithms, problems with large data sets, etc.)

How Fault Tolerance is achieved?

- Master delegates execution of the service to each slaves
- A slave may perform same task
- Slaves are kept in synch, so if one of them fails, the system keeps running
- Reliability is achieved through replication

Computational Accuracy?

- Slaves are not identical
- Requires at least three slaves
- Each slave provides a different implementation of the service (may be using different algorithms)
- Master delegates execution of the service to each slave
- Master compares results from slaves
- If results agree, return to client
- If results disagree, take appropriate action
  - Generate exception
  - Have slaves "vote" (return most common result)

Master-slave

- Applicable Domains
  - Software systems where reliability is critical.
- Example - Flight Control Systems (Aerospace)System
  - Avionics computers in airplanes use a triple-modular redundancy (TMR) system.
  - Process: Three independent systems (slaves) calculate the same flight path.
  - The master (controller) compares the results
    - If all agree- The result is accepted.
    - If one disagrees- The two matching results are accepted (majority voting).
  - Purpose: To ensure the system continues operating even if one subsystem fails.

Layered Architecture

- Structure: Divides the system into multiple layers based on abstraction levels.
- Use Case: Web applications (UI, Service, Data layers).

Layered Architecture

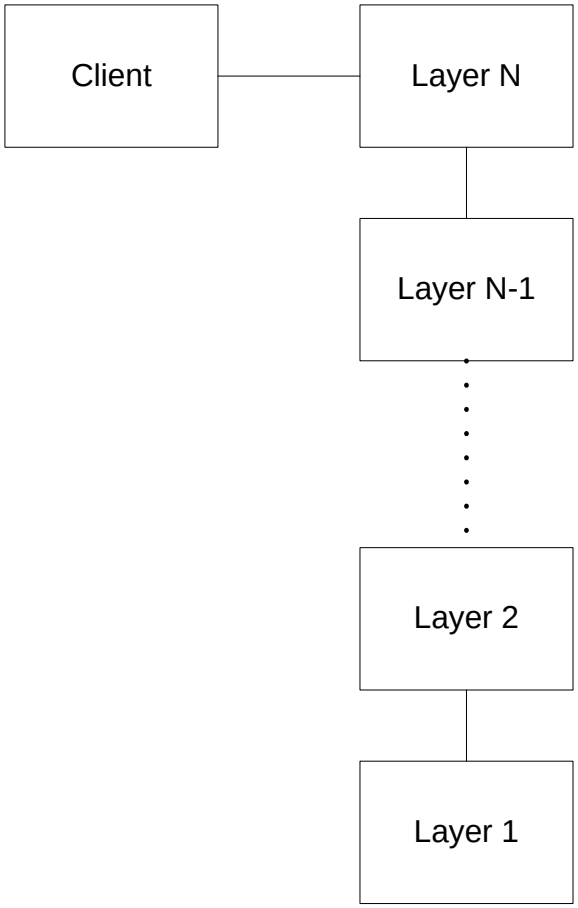
- The system is decomposed into a number of higher and lower layers in a hierarchy.
- Each layer provides a related set of services.
- Also, each layer has its own sole responsibility in the system.

Layered Architecture

- **Layer 1** is at the bottom and contains components closest to the hardware/OS.
- **Layer N** is at the top and contains components that interact directly with the system's clients.
- A request to layer i +1 invokes the services provided by the layer i via the interface of layer i.
- The response may go back to the layer i +1 if the task is completed; otherwise layer i continually invokes services from the layer i -1 below.

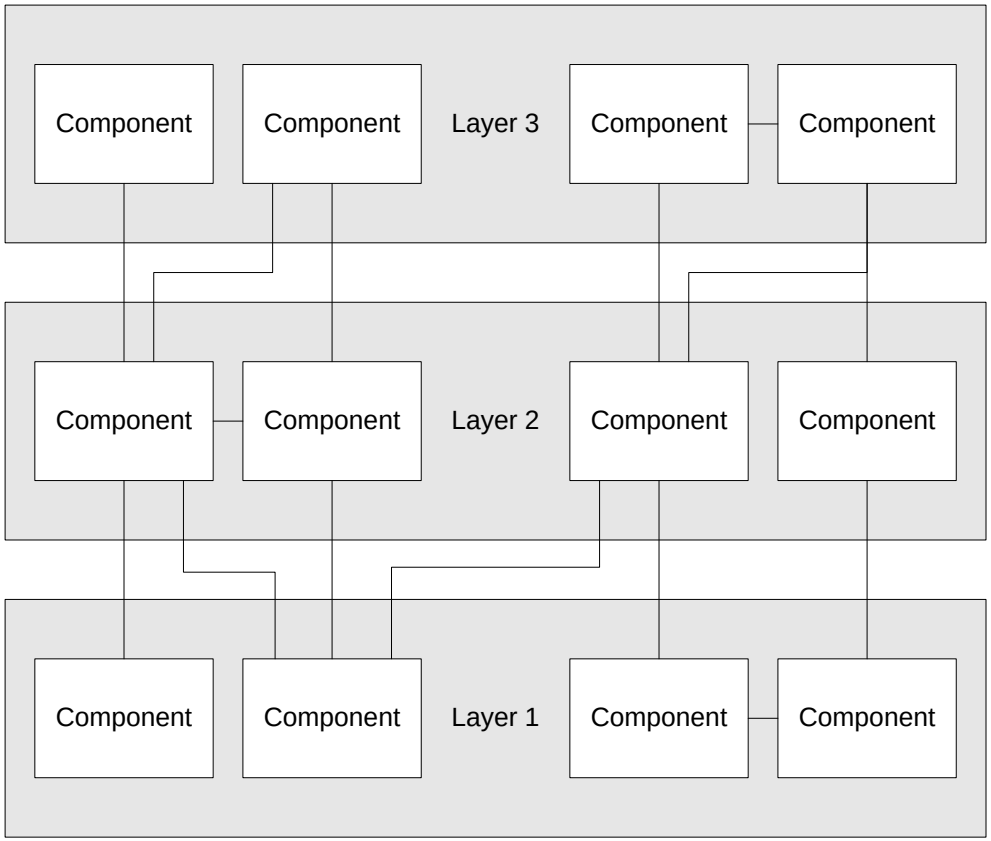
Layered Architecture

- **Layer j** uses the services of **Layer j-1** and provides services to **Layer j+1**
- Most of Layer j's services are implemented by composing Layer j-1's services in meaningful ways
- Layer j raises the level of abstraction by one level



Structure

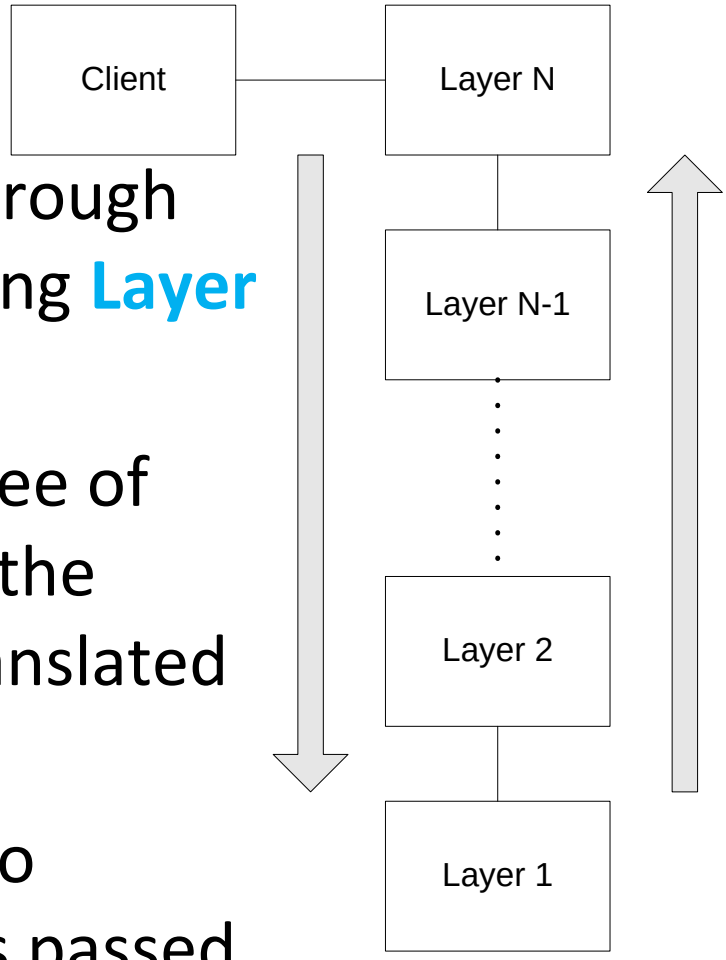
- **Layer j's** services are used by **Layer j+1**
- Components in Layer j may also use each other
- There are no other direct dependencies between layers



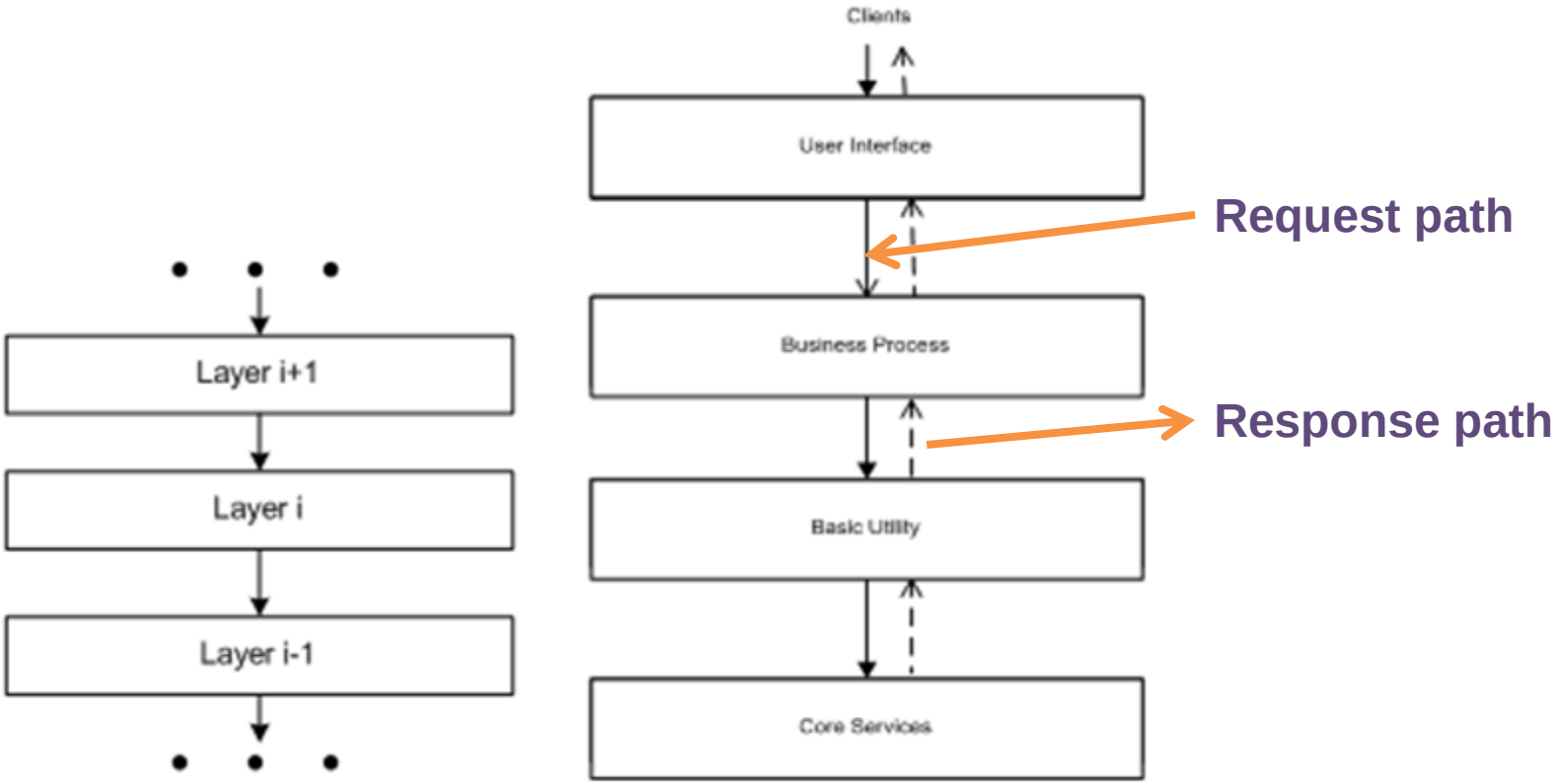
Components are classes and methods

Dynamic Behavior

- **Scenario I: (Common case)**
- Request at **Layer N** travels down through each successive layer, finally reaching **Layer 1.**
- The top-level request results in a tree of requests that travel down through the layers (each request at Layer j is translated into multiple requests to Layer j-1).
- Results from below are combined to produce a higher-level result that is passed to the layer above.

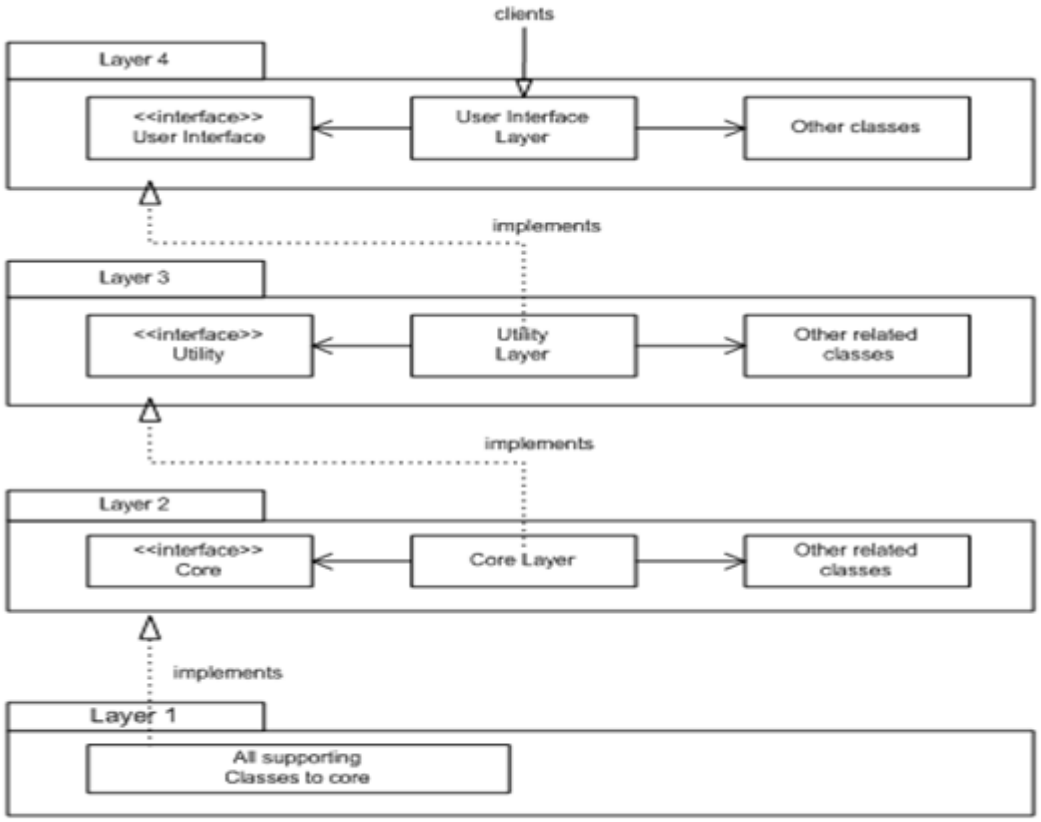


Dynamic Behavior



A partial layered architecture; business example

Dynamic Behavior



Component-based layered architecture

Each layer gets support from its lower adjacent layer by an interface implementation and related classes in the same layer

Top level deals with user interface, next level utilities and core services

Implementation

- Determine the **number of layers** (i.e., abstraction levels).
- Name the layers and **assign responsibilities** to them.
- Define the **interface** for each layer

Implementation

Error handling strategy

- Part of a layer's **interface** is the **set of errors** it might return
- The errors returned by a layer should match its level of abstraction
- Errors received from the **layer below** should be **mapped** into higher-level errors that are appropriate for the **layer above**
- **Low-level** errors should not be allowed to "leak out" and become visible to **high-level layers**

Relaxed Layers

- **Layer j** can call directly into **any layer below** it, not just **Layer j-1**

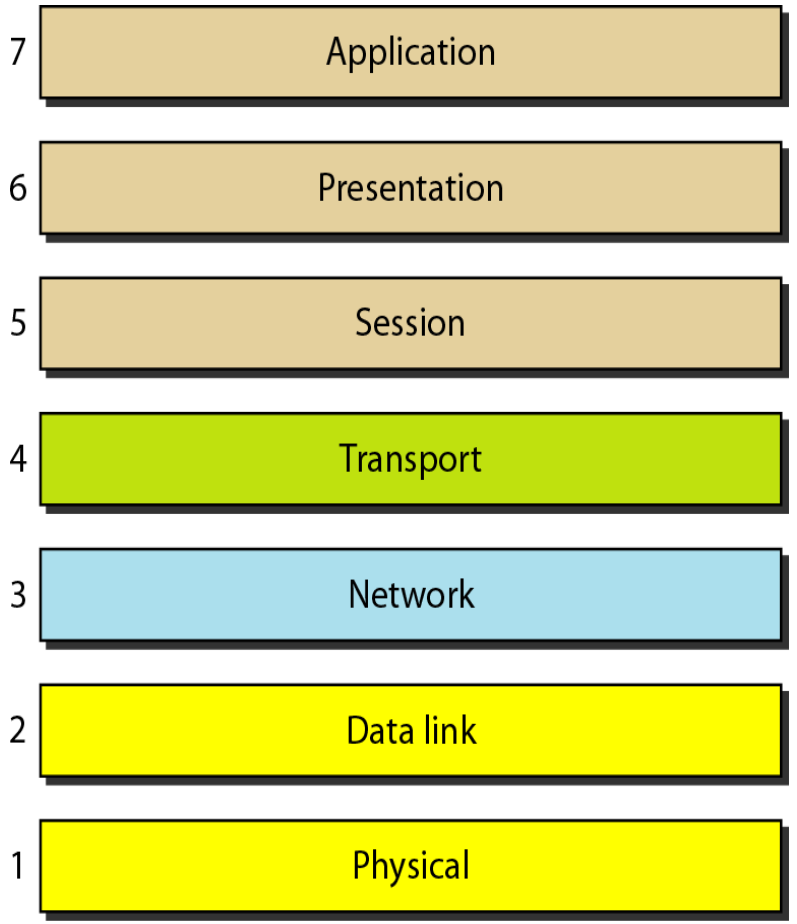
- Pros
  - More flexible and efficient than strict layers
  - Easier to build than strict layers

- Cons
  - Less understandable and maintainable than strict layers

Known Uses: International Standard Organization (ISO) Reference Model

This model used for data communications

Application Layer (All)
Presentation Layer (People)
Session Layer (Seems)
Transport Layer (To)
Network Layer (Need)
Data Link Layer (Data)
Physical Layer (Processing)



Benefits of Layered Architecture

- Incremental Development
- Clear Separation of Concerns
- Portability and Component Reusability

Limitations of Layered Architecture

- Performance Overhead
- Deadlocks and Bridging Issues
- Complex Error Handling

Virtual Machine Architecture

- Software implementation of hardware or OS.
- Use Case: JVM, .NET CLR.
- Example: Running older apps on Windows via VM.

Virtual Machine Example: JVM

- JVM executes Java bytecode across different platforms.
- Provides platform independence.